



Министерство науки и высшего образования Российской
Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана (национальный
исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

Факультет «*фундаментальных наук*»

Кафедра «*математического моделирования*»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к научно-исследовательской работе

на тему:

ПОДХОД LARGE NEIGHBORHOOD SEARCH В ЗАДАЧЕ 3D BIN PACKING

Студент группы ФН12-81Б

(подпись, дата) *Иртюга И. В.*

Руководитель НИР

(подпись, дата) *Фетисов Д. А.*

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана (национальный
исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ

Заведующий кафедрой _____

« ____ » _____ 20__ г.

ЗАДАНИЕ
на выполнение научно-исследовательской работы

по теме

«Подход Large Neighborhood Search в задаче 3D Bin Packing»

Студент группы ФН12-81Б

Иртюга Илья Владимирович

(Фамилия, Имя, Отчество)

Направленность НИР исследовательская

(учебная, исследовательская, практическая, производственная и др.)

Источник тематики кафедра

График выполнения работы: 25% к ____ нед., 50% к ____ нед., 75% к ____ нед., 100% к ____ нед.

Задание _____

Оформление научно-исследовательской работы:

Расчетно-пояснительная записка на ____ листах формата А4.

Перечень графического (иллюстративного) материала _____

Дата выдачи задания « ____ » _____ 20__ г.

Руководитель НИР

_____ *Фетисов Д. А.*
(подпись, дата)

Студент

_____ *Иртюга И. В.*
(подпись, дата)

Примечание. Задание оформляется в двух экземплярах: один выдается студенту, второй хранится на кафедре.

СОДЕРЖАНИЕ

1. Введение	4
2. Постановка задачи	4
2.1. Greedy как базовый метод	5
3. Метод Large Neighborhood Search	5
3.1. Общая идея LNS	5
3.2. Этапы работы алгоритма	6
3.3. Разрушение и восстановление решения	6
3.4. Критерий принятия решения	6
4. Реализация LNS в проекте	7
4.1. Общая структура реализации	7
4.2. Начальное решение	7
4.3. Destroy-эвристики и стабилизация	7
4.4. Процедура repair	8
4.5. Ключевые параметры	8
5. Экспериментальное сравнение LNS и Greedy	8
5.1. Данные для сравнения	8
5.2. Итоговый score	9
5.3. Компоненты метрики	9
5.4. Интерпретация результатов	10
6. Выводы	11
7. Список литературы	12

1. Введение

Задача трёхмерной упаковки прямоугольных объектов в ограниченный контейнер относится к классу сложных комбинаторных задач и имеет прямое прикладное значение для складской логистики, паллетизации и транспортировки грузов [1]. В практических приложениях важно не просто разместить как можно больше коробок, но и соблюдать ограничения по массе, устойчивости, ориентации и хрупкости товара.

В рассматриваемом проекте исследуется single-pallet постановка задачи 3D Bin Packing. Для неё уже была выполнена курсовая работа, в которой кратко сравнивались алгоритмы решения. В настоящей НИР внимание сосредоточено только на одном методе — *Large Neighborhood Search* (LNS). Именно этот подход показывает лучшие результаты на общих наборах данных проекта по итоговой метрике качества.

Цель работы — исследовать применение подхода Large Neighborhood Search к задаче 3D Bin Packing и объяснить, почему данный метод превосходит базовый greedy-алгоритм.

Для достижения цели решаются следующие задачи:

1. формализовать постановку задачи и используемую метрику качества;
2. кратко описать greedy как базовый метод;
3. подробно разобрать структуру и параметры LNS-алгоритма;
4. рассмотреть реализацию LNS в проекте;
5. сравнить LNS и greedy по имеющимся экспериментальным результатам;
6. сформулировать выводы о достоинствах и ограничениях подхода.

Объект исследования — алгоритмы укладки прямоугольных коробок на паллету.

Предмет исследования — метаэвристический подход Large Neighborhood Search и его практическое применение к задаче 3D Bin Packing.

2. Постановка задачи

Каждая задача в проекте задаётся в формате JSON и включает описание одной паллеты и набора типов коробок. Для паллеты указываются длина, ширина, максимальная высота укладки и допустимая масса. Для каждого типа коробки задаются размеры, масса, количество экземпляров и логические признаки `strict_upright`, `fragile`, `stackable`.

Цель алгоритма состоит в том, чтобы построить допустимую укладку коробок внутри паллеты. При этом должны выполняться следующие ограничения:

1. каждая коробка целиком находится внутри габаритов паллеты;
2. коробки не пересекаются по объёму;
3. суммарная масса не превышает ограничение паллеты;

4. для коробок с признаком `strict_upright` запрещён поворот, меняющий исходную высоту;
5. если коробка расположена выше пола паллеты, не менее 60% площади её основания должно иметь опору;
6. тяжёлые коробки не должны располагаться над хрупкими без штрафа по метрике.

Качество решения определяется формулой

$$\text{score} = 0.50 \cdot U_{\text{vol}} + 0.30 \cdot C_{\text{items}} + 0.10 \cdot F_{\text{frag}} + 0.10 \cdot T_{\text{time}},$$

где U_{vol} — коэффициент заполнения объёма паллеты, C_{items} — доля размещённых объектов, F_{frag} — оценка по хрупкости, T_{time} — временной балл.

Из этой формулы видно, что главными компонентами качества являются заполнение контейнера и полнота размещения. Именно поэтому для рассматриваемой задачи особенно важен метод, умеющий улучшать уже найденную укладку, а не только быстро строить первое допустимое решение.

2.1. Greedy как базовый метод

Жадный алгоритм используется как baseline и как основа для старта LNS. Он разворачивает количества коробок в последовательность экземпляров, сортирует их по приоритету и затем поочерёдно пытается разместить каждый объект в допустимую позицию. В качестве кандидатов используются *extreme points*, а положение коробки уточняется по схеме “gravity drop”, то есть с опусканием на минимально возможную высоту.

Преимущество greedy-подхода состоит в скорости и простоте. Недостаток состоит в локальности: если неудачное размещение выполнено на раннем шаге, алгоритм уже не может вернуться назад и перестроить структуру укладки. В результате ухудшается использование свободного пространства и уменьшается доля размещённых коробок.

3. Метод *Large Neighborhood Search*

3.1. Общая идея LNS

Подход *Large Neighborhood Search* относится к метаэвристикам крупноокрестного поиска [2]. Его основная идея состоит в том, что решение улучшается не малыми локальными перестановками, а за счёт разрушения значимой части текущей конфигурации и последующего восстановления этой части более удачным образом.

Для задач упаковки такой подход особенно естественен. Если несколько коробок были неудачно размещены в начале, они могут заблокировать большой объём пространства. Тогда локальная правка одной позиции уже не помогает, а частичная перестройка решения позволяет освободить полезные области паллеты и перераспределить объекты более компактно.

3.2. Этапы работы алгоритма

В проекте LNS работает по следующей схеме:

1. строится начальное жадное решение;
2. выбирается часть текущей укладки, которая удаляется;
3. образовавшееся частичное решение стабилизируется;
4. удалённые и непомятые объекты заново вставляются процедурой восстановления;
5. новое решение оценивается и либо принимается, либо отвергается.

Такая структура позволяет сочетать преимущества двух подходов: greedy быстро даёт допустимый старт, а LNS затем выходит за пределы локально жадного выбора.

3.3. Разрушение и восстановление решения

На этапе разрушения из текущей укладки удаляется часть коробок. В реализации проекта используются разные destroy-эвристики: случайное удаление, удаление верхних коробок, удаление кластера около случайной коробки, удаление элементов, связанных с нарушениями хрупкости, и полный рестарт. Число удаляемых объектов зависит от текущего размера укладки и параметров конфигурации.

После destroy-фазы вызывается стабилизация частичного решения: если какие-то коробки потеряли опору и стали “зависшими”, они тоже удаляются. Это делает промежуточное состояние геометрически согласованным.

Далее запускается этап repair. В текущей реализации он основан на beam search. Алгоритм рассматривает несколько лучших частичных состояний, для каждого перебирает допустимые повороты и extreme points, проверяет пересечения, опору и ограничения по хрупкости, после чего оставляет наиболее перспективные варианты для дальнейшего расширения. Благодаря этому восстановление оказывается заметно сильнее, чем одношаговый greedy-выбор.

3.4. Критерий принятия решения

После восстановления укладка оценивается той же целевой функцией, что и финальный ответ. Если новое решение лучше лучшего найденного ранее, оно сохраняется. Кроме того, в проекте используется simulated annealing: часть ухудшений может быть принята как текущее рабочее состояние с вероятностью, зависящей от температуры. Это позволяет алгоритму выходить из локальных максимумов.

Если на протяжении нескольких итераций улучшения не происходит, включается механизм перезапуска поиска. Он формирует новую рабочую точку из лучшего найденного решения, но с другой перестановкой очереди объектов. Такая комбинация LNS и управляемых рестартов повышает устойчивость метода.

4. Реализация LNS в проекте

4.1. Общая структура реализации

Реализация LNS тесно связана с greedy-алгоритмом: используются общие процедуры генерации допустимых поворотов, проверки опоры, хрупкости и пересечений, а также пространственная структура данных, ускоряющая геометрические проверки. Поэтому LNS не дублирует базовую геометрию, а опирается на уже согласованные механизмы.

Основные смысловые блоки реализации таковы:

1. конфигурации параметров алгоритма;
2. процедура построения начального решения;
3. функции вычисления итоговой и частичной оценки;
4. набор destroy-эвристик;
5. процедуры восстановления решения;
6. главный цикл оптимизации.

4.2. Начальное решение

Начальное решение строится жадно, но не единственным запуском. Сначала применяется базовая сортировка товаров, затем выполняется несколько детерминированных вариантов упорядочивания, а после этого — серия случайных greedy-рестартов в пределах выделенного временного бюджета. Итогом фазы становится лучший жадный результат, найденный на старте.

Это важная особенность реализации: LNS стартует не с произвольной укладки, а с уже усиленного greedy-решения. Тем самым сокращается расстояние до качественного ответа и повышается эффективность последующего поиска.

4.3. Destroy-эвристики и стабилизация

В реализации используются несколько destroy-операторов. Для single-pallet постановки особенно важны следующие идеи:

1. удаление верхних коробок позволяет перестроить верхние уровни и лучше использовать высоту;
2. удаление кластера позволяет локально перепланировать плотную область;
3. удаление коробок, близких по объёму к непомещённым, освобождает место для более выгодной перестановки;
4. полный рестарт помогает выйти из тупиковой конфигурации.

После разрушения вызывается процедура стабилизации, которая каскадно удаляет коробки без достаточной опоры. Благодаря этому beam search всегда восстанавливает уже согласованное частичное решение.

4.4. Процедура repair

Процедура восстановления является центральной частью всей реализации. Её основу составляет beam search. Алгоритм хранит не полные копии укладки, а только изменения относительно фиксированной части, то есть *delta-state*. Это уменьшает накладные расходы при поиске.

Для каждого луча beam search вычисляется частичная оценка

$$\text{partial_score} = 0.50 \cdot U_{\text{vol}} + 0.30 \cdot C_{\text{items}} - 0.005 \cdot n_{\text{frag_viol}},$$

которая ориентирует поиск на лучшее заполнение объёма и большее число размещённых объектов при умеренном штрафе за проблемы хрупкости.

Дополнительно восстановление выполняется не один раз: очередь рассматриваемых объектов несколько раз случайно переставляется, и из нескольких запусков выбирается лучший результат. Таким образом, repair сочетает направленный поиск и стохастическое разнообразие.

4.5. Ключевые параметры

Наиболее важные параметры основной конфигурации LNS приведены в таблице 1.

Таблица 1. Ключевые параметры основной конфигурации LNS

Параметр	Значение	Блок	Смысл
time_budget_s	4.8	поиск	общий бюджет времени
greedy_phase_s	1.5	старт	бюджет на greedy-рестарты
destroy_k_fraction	0.15	destroy	доля удаляемых коробок
beam_width	8	repair	число лучей beam search
repair_queue_limit	50	repair	размер очереди на восстановление
repair_restarts	2	repair	число дополнительных запусков восстановления
early_stop_patience	25	управление	порог остановки по застою
ils_kicks	5	управление	число перезапусков поиска
sa_initial_temp	0.015	приём	стартовая температура
sa_cooling_rate	0.92	приём	коэффициент охлаждения

5. Экспериментальное сравнение LNS и Greedy

5.1. Данные для сравнения

Для анализа использованы сохранённые результаты вычислительных экспериментов для greedy и LNS. Рассматриваются три общих набора задач:

1. dataset_100.json — 100 задач;
2. dataset_200_2.json — 200 задач;

3. `dataset_benchmark.json` — 200 задач.

Для построения графиков в проект НИР включён воспроизводимый скрипт, который обрабатывает сохранённые результаты и формирует сводные рисунки и таблицы.

5.2. Итоговый score

Средние значения итогового **score** приведены в таблице 2.

Таблица 2. Сравнение LNS и greedy по итоговому score

Набор	Greedy	LNS	Δ	Преимущество LNS
<code>dataset_100</code>	0.684	0.711	+0.027	устойчивое
<code>dataset_200_2</code>	0.712	0.743	+0.031	устойчивое
<code>dataset_benchmark</code>	0.655	0.730	+0.075	сильное

Во всех трёх случаях LNS превосходит базовый greedy-алгоритм. Особенно заметен выигрыш на `dataset_benchmark`, где задачи сложнее и пространство возможных перестроек богаче. Это подтверждает, что крупноокрестностный поиск особенно полезен тогда, когда одной жадной проходки уже недостаточно.

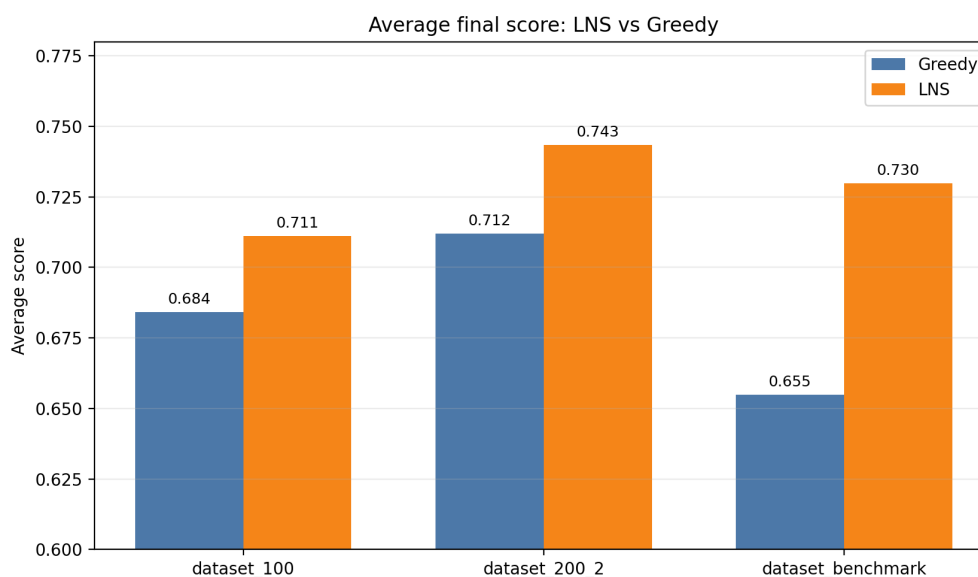


Рис. 5.1. Средний итоговый score для LNS и greedy

5.3. Компоненты метрики

Для понимания причин превосходства полезно посмотреть на компоненты метрики. Наиболее показателен набор `dataset_benchmark.json`; результаты приведены в таблице 3.

Таблица 3. Компоненты метрики на dataset_benchmark

Метод	Score	Volume	Coverage	Fragility	Time
Greedy	0.6549	0.6155	0.4906	1.0000	1.0000
LNS	0.7298	0.7172	0.6036	0.9833	0.9175

LNS немного уступает greedy по времени и совсем незначительно по компоненте хрупкости, однако существенно выигрывает по двум главным составляющим: использованию объёма и доле размещённых объектов. Именно эти улучшения и дают основной прирост итогового балла.

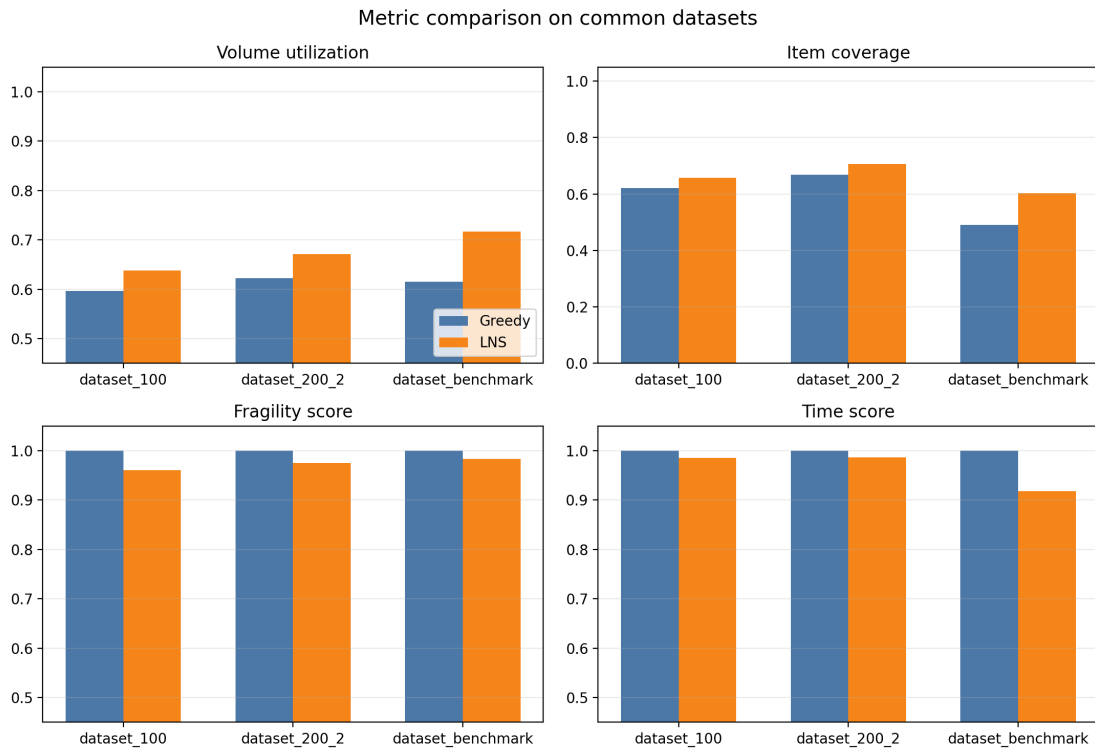


Рис. 5.2. Сравнение компонент метрики на общих наборах

5.4. Интерпретация результатов

Преимущество LNS над greedy объясняется структурно:

1. greedy принимает решения только один раз, а LNS умеет пересматривать уже построенную укладку;
2. частичное разрушение решения помогает убрать последствия плохого раннего выбора;
3. beam search на этапе repair сильнее использует освободившееся пространство;
4. simulated annealing позволяет не застревать в локально хороших, но глобально слабых конфигурациях.

Именно поэтому LNS лучше заполняет паллету и чаще размещает больше коробок, чем baseline. При этом потеря по времени оказывается умеренной: среднее время решения растёт, но остаётся в разумных границах для заданной метрики.

6. Выводы

В работе была рассмотрена single-pallet постановка задачи 3D Bin Packing и подробно исследован подход Large Neighborhood Search. Показано, что данная реализация строится поверх жадного начального решения и усиливает его за счёт повторных перестроек укладки, направленных destroy-операторов, beam-search восстановления и критерия принятия на основе simulated annealing.

Сравнение с greedy-алгоритмом показало, что LNS стабильно выигрывает по итоговому score на всех общих наборах данных проекта. Главная причина состоит в том, что метод лучше использует свободный объём паллеты и размещает большую долю коробок. Таким образом, LNS оказывается наиболее эффективным подходом для данной постановки задачи.

Ограничения метода связаны прежде всего с большей сложностью настройки и более высоким временем работы по сравнению с greedy. Тем не менее в рамках рассматриваемого проекта этот рост времени оказывается оправданным, поскольку улучшение ключевых компонент метрики значительно перекрывает потери по временной составляющей.

7. Список литературы

1. Martello S., Pisinger D., Vigo D. The Three-Dimensional Bin Packing Problem // *Operations Research*. 2000. Vol. 48. No. 2. P. 256–267. DOI: <https://doi.org/10.1287/opre.48.2.256.12386>.
2. Pisinger D., Røpke S. Large Neighborhood Search // *Handbook of Metaheuristics*. 2nd ed. Springer, 2010. P. 399–420.
3. Faroe O., Pisinger D., Zachariasen M. Guided Local Search for the Three-Dimensional Bin-Packing Problem // *INFORMS Journal on Computing*. 2003. Vol. 15. No. 3. P. 267–283.
4. Статья из журнала Института системного анализа РАН [Электронный ресурс]. URL: http://www.isa.ru/aidt/2016-03/31_43.pdf (дата обращения: 27.04.2026).